How to deploy: shared login and cheering for CheerIT and ScoreIT

Follow it from top to bottom. Each step says where to run it, what to run, what you should see, and what to do if you don't.

Updated 2 Nov 2026

Start here. Follow this page from top to bottom. Do not skip steps. Every step tells you:

- **Where** to run it (which server, which folder)
- **Run** (the exact commands)
- **You should see** (how to know it worked)
- **If not** (what to do)

Nothing secret is written on this page. You will type passwords and keys yourself. Never paste them into chat, tickets or Git.

---

1. What you are deploying (in plain words)

Goal 1: one login for all apps. A person who logs in to ScoreIT, GameIT, PredictIt or TurfIT with their phone number is stored **once**, in a new small database called the **identity database**. Each app keeps its own data and only gets one extra empty column called `identity_id`.

Goal 2: cheer from ScoreIT, save in CheerIT. On a ScoreIT match page, a viewer records a photo or video. It is saved in CheerIT as a normal cheer, under their CheerIT account, **without asking for another login** (this is called SSO).

The pieces:

- **Identity database** `cheeritidentity`: one row per person. Holds no app data.
- **Identity service**: a small program (port 4100) that is the only thing allowed to touch that database.
- **The apps**: ScoreIT, GameIT, PredictIt, TurfIT (and Sponsor, optional). After every OTP login they tell the identity service who logged in. **If the service is down, logins still work.**
- **CheerIT API**: gets one small addition so it can accept the SSO login.
- **ScoreIT**: creates the CheerIT events, sends scores, and shows the cheer card.

2. Words used in this file

- **pm2**: the program that keeps each backend running. `pm2 restart <name>` restarts one.
- `.env` : the file with an app's settings. It is in the app's folder. Each line is `NAME=value` .
- `<something>` : a placeholder in angle brackets. **You must replace it with the real value.** Never type the brackets.
- **Backend server**: the Linux server where the apps run under pm2.
- **Dry run**: a test run that only reads and writes nothing.
- **Key**: a password for one app to talk to the identity service. It is shown **once**.

3. Six safety rules

1. **Back up every database first.**
2. **Test only with phone numbers you own.** Every login test sends a real SMS.
3. **Never paste a `<placeholder>` .** Replace it first.
4. **Never run `prisma db push` or `prisma migrate` on an existing database.** Only `npx prisma generate` . (Exception: a brand-new empty database, in Part 5.)
5. **One app at a time.** Check login works before starting the next.
6. **Keys are shown once.** Issuing a new key for an app cancels its old key. Write the key straight into the `.env` using the commands in this file.

Two habits that prevent mistakes: do not put a `# comment` after a command on the same line, and always check a `.env` for **repeated lines** (the last one wins).

4. Before you start: collect these values

Write them down. You will need them in every part.

- **Backend server:** how you log in (SSH), and the folder where apps live.
- **Database server:** host name, port (usually `5432`), and an admin user.
- **Database names:** ScoreIT, GameIT, PredictIt, TurfIT, (Sponsor), and CheerIT.
- **Identity database name:** use `cheeritidentity` .
- **CheerIT API address** (https, no `/api` at the end), for example `https://devcheerback.avenengage.com` .
- **ScoreIT address**, for example `https://devscoreit.avenengage.com` .
- **CheerIT upload bucket** name and its region (dev uses `ap-south-1`).

- **An organization id** from CheerIT's `Organization` table (used to create the "ScoreIT" competitions).
- **A phone number you own** for testing.

Check the server has: Node 22 or newer (`node -v`), `pm2` , `git` , `psql` .

PART 1. Security items (do these first)

Tick each one.

- [ ] **ScoreIT login fix.** Today any code logs in to ScoreIT as any phone number. The fix is in branch `feat/cheerit-cheer` . You verify it in Part 3, step 3.3. This is the most urgent item.

- [ ] **Committed `.env` files.** Several repositories contain a `.env` in Git, including CheerIT's (commit "env publish", with real production secrets). Treat everything in those files as leaked: **change the secrets**, then stop tracking the file with `git rm --cached .env` and add `.env` to `.gitignore` .
  - **Careful:** a server that pulls that change can lose its own `.env` . On each server first run `git update-index --skip-worktree .env` , or copy the file aside and put it back after the pull.

- [ ] **CheerIT has open routes.** Anyone on the internet can create events, teams and venues in CheerIT. Ask CheerIT's team to add a login check.

- [ ] **CheerIT upload route** trusts the `user_id` sent by the browser. Tell CheerIT's team.

- [ ] **Turn off OTP logging in production.** ScoreIT: `LOG_OTP_TO_CONSOLE=false` . PredictIt: `OTP_DEV_MODE=false` and `OTP_LOG_CODE=false` . Check GameIT and TurfIT the same way.

- [ ] **Sponsor:** make sure `AUTH_SECRET` is set. Without it Sponsor uses a built-in secret and logins can be forged.

- [ ] **Change any secret that was ever pasted into a chat:** database passwords, AWS keys, SMS key, JWT secrets, identity keys.

PART 2. Identity database and service

Where: the backend server.

Step 2.1: Back up every database

```SHELL
mkdir -p ~/identity-backups
for db in <score_db> <gameit_db> <predictit_db> <turfit_db> <cheerit_db>; do PGPASSWORD='<admin-password>' pg_dump -h <db-host> -U <admin-user> -Fc "$db" -f ~/identity-backups/$db-$(date +%Y%m%d-%H%M).dump; done
ls -la ~/identity-backups
```

You should see: one file per database, none empty. **To restore later:** `pg_restore --clean -h <db-host> -U <admin-user> -d <db> <file>`

Step 2.2: Get the identity code and create its database

```SHELL
git clone https://github.com/kartikavencheer/cheeritidentity.git
cd cheeritidentity
git checkout main
npm install
export PGPASSWORD='<admin-password>'
psql -h <db-host> -U <admin-user> -d postgres -c "CREATE DATABASE cheeritidentity;"
psql -h <db-host> -U <admin-user> -d cheeritidentity -f sql/001_identity_schema.sql
psql -h <db-host> -U <admin-user> -d cheeritidentity -f sql/002_app_clients.sql
psql -h <db-host> -U <admin-user> -d cheeritidentity -c "\dt"
```

You should see: exactly these 5 tables: `app_clients`, `migration_log`, `migration_runs`, `user_apps`, `users`.

If not: do not continue. Do not run any file named `003_*`.

Step 2.3: Write the identity `.env`

> **Where:** the `cheeritidentity` folder. Open it with `nano .env` so passwords are not saved in your shell history. Leave a line empty to skip that app.

SHELL

```
IDENTITY_URL=postgresql://<identity-user>:<pw>@<db-host>:5432/cheeritidentity
PORT=4100

SRC_SCORE_URL=postgresql://<readonly-user>:<pw>@<db-host>:5432/<score_db>
SRC_BOOKING_URL=postgresql://<readonly-user>:<pw>@<db-host>:5432/<gameit_db>
SRC_PREDICTIT_URL=postgresql://<readonly-user>:<pw>@<db-host>:5432/<predictit_db>
SRC_TURFIT_URL=postgresql://<readonly-user>:<pw>@<db-host>:5432/<turfit_db>
SRC_SPONSOR_URL=
SRC_COM_URL=

LINK_SCORE_URL=postgresql://<owner-user>:<pw>@<db-host>:5432/<score_db>
LINK_BOOKING_URL=postgresql://<owner-user>:<pw>@<db-host>:5432/<gameit_db>
LINK_PREDICTIT_URL=postgresql://<owner-user>:<pw>@<db-host>:5432/<predictit_db>
LINK_TURFIT_URL=postgresql://<owner-user>:<pw>@<db-host>:5432/<turfit_db>
LINK_SPONSOR_URL=
LINK_COM_URL=
```

What the lines mean:

- `SRC_*` = **read-only** login to each app's database (used to read users).
- `LINK_*` = the **owner** login of each app's database (used only to add and fill the `identity_id` column).
- `*_COM_*` = the CheerIT database. **Leave empty until Part 4 is done** and CheerIT's database has been backed up.
- `IDENTITY_URL` must be a **different** database from every `SRC_*`.
- Add `?sslmode=require` to a URL if that database needs SSL.

Note: the `.env` in this repository is tracked by Git and holds local development addresses. On a server, create your own and run `git update-index --skip-worktree .env`.

Step 2.4: Dry run (reads only)

SHELL

```
node --env-file=.env src/dry-run.ts
```

You should see: one line `Reading <app> from … (read-only)` for each app you configured, then `N records → M people`. On the dev server it was 42 records to 16 people.

If not: if you see `Skipped` for an app, its `SRC_*` line is wrong or empty. If an app shows 0 records, its database or table is wrong. Fix it first.

Open the report folder it prints (`out/<time>/`). These notes are normal:

- `name_conflict`: harmless spelling differences.
- `same_app_multiple_roles`: by design (one person, two roles).
- `invalid_phone`: placeholder numbers such as `9999999999` are left out. Those users keep working.

Stop here and get the numbers approved before Step 2.5.

Step 2.5: Write the people

```shell
node --env-file=.env src/apply.ts
node --env-file=.env src/apply.ts --apply
node --env-file=.env src/apply.ts --verify-only
node --env-file=.env src/apply.ts --apply
```

You should see: after the first `--apply`: `Written: N new people` and six `PASS` lines. After the **second** `--apply`: `0 new people` and `0 new memberships`.

If not: any `FAIL` or "ownership clash": stop and investigate.

Step 2.6: Link each app database

This is the first write into the app databases. It adds the empty column `identity_id` and fills it.

```shell
node --env-file=.env src/link.ts
node --env-file=.env src/link.ts --apply
```

```
node --env-file=.env src/link.ts --verify-only
node --env-file=.env src/link.ts --apply
```

You should see: for each app `Written: column ensured, N rows linked` and `Verification PASS … 0 mismatched`. The second `--apply` links 0 new rows.

If not: `NOT IN IDENTITY: 1` is normal for a placeholder-phone user or one new signup (run `apply.ts` again). If an app refuses because a row already holds a different identity, do not override it. Investigate.

Step 2.7: Start the identity service

SHELL

```
cd <path>/cheeritidentity
pm2 start "npm start" --name identity
pm2 save
curl http://127.0.0.1:4100/health
```

You should see: `{"ok":true}`

Make pm2 survive a reboot **once per server**. First check: `systemctl is-enabled pm2-<your-user>`. If it says `enabled`, do nothing. If not, run `pm2 startup` and then run the line it prints.

Where the service listens:

- Everything on **one server:** use `IDENTITY_SERVICE_URL=http://127.0.0.1:4100`. This is the safest.
- A backend (including the CheerIT API) on **another server:** start the service with `HOST=<private-ip>` and open port 4100 **only** to those servers. Never to the internet.

PART 3. Connect each app (one at a time)

Order: ScoreIT first (it has the security fix), then PredictIt, TurfIT, GameIT. Sponsor last, optional.

3.1 The apps

ScoreIT

- Repository: `Shriya2020/ScoreIT`
- Branch: `feat/cheerit-cheer` (has the login fix and the cheer feature)
- Folder: `ScoreIT/backend` (the website is `ScoreIT/frontend`)
- pm2 name: `scoreit-backend`
- Key name: `score`

PredictIt

- Repository: `Shriya2020/predictitapp`
- Branch: `master`
- Folder: `predictitapp/backend`
- pm2 name: `predictit-backend`
- Key name: `predictit`

TurfIT

- Repository: `Shriya2020/turfitweb`
- Branch: `master` plus `fix/identity-skip-data-avatar`
- Folder: `turfitweb/Backend`
- pm2 name: `turfitweb-backend`
- Key name: `turfit`

GameIT

- Repository: `Shriya2020/pickleball`
- Branch: `main` plus `fix/identity-skip-data-avatar`
- Folder: `pickleball/backend`
- pm2 name: `pickleball-backend`
- Key name: `booking`

Sponsor (optional)

- Repository: `Shriya2020/CheerIT-Sponsor`

- Branch: `stackchnage` (spelled like this)
- Folder: `CheerIT-Sponsor/backend`
- pm2 name: `cheerit-sponsor-backend`
- Key name: `sponsor`

Production folders and names can differ. Find an app's folder with: `pm2 describe <pm2-name> | grep "exec cwd"`.

The avatar fix stops GameIT and TurfIT from sending big base64 profile photos to the identity service (it accepts 10 KB, their photos are up to 300 KB, which caused an HTTP 500).

3.2 The same recipe for every app

Step A. Deploy the app's code (merge its branch the way your team normally does).

Step B. Install

```shell
cd <backend-folder>
npm install
npx prisma generate
```

Only `generate`. Never `migrate` or `db push`. Sponsor has no Prisma: skip that last line.

Step C. Issue a key and write it into the app's `.env`

Replace `<key-name>` and `<backend-folder>` with the app's values. The commands put the key in the file without ever showing it.

```shell
cd <path>/cheeritidentity
KEY=$(npm run -s issue-key -- <key-name> | grep -o 'cid_<key-name>_[A-Za-z0-9_-]*')
echo "key length: ${#KEY}"
cd <backend-folder>
grep -c '^IDENTITY_' .env
echo "" >> .env
echo "IDENTITY_SERVICE_URL=http://127.0.0.1:4100" >> .env
echo "IDENTITY_API_KEY=$KEY" >> .env
unset KEY
```

> **You should see:** key length about **50 to 57**, and the `grep -c` line prints `0`.

> **If not:** a length of `0` means the key command failed: stop and read its error. If `grep -c` prints more than 0, the lines already exist. Do not add them again. Replace instead: `sed -i "s|^IDENTITY_API_KEY=.*|IDENTITY_API_KEY=$KEY|" .env`.

Step D. Restart

```shell
pm2 restart <pm2-name> --update-env
pm2 logs <pm2-name> --lines 30 --nostream
```

Step E. Test with your own phone number

1. Log in on that app's website (a real OTP SMS is sent).
2. Check it was recorded:

```shell
PGPASSWORD='<pw>' psql -h <db-host> -U <identity-user> -d cheeritidentity -c "select u.phone,
a.app_key, a.last_seen_at from users u join user_apps a on a.user_id=u.id where
u.phone='+91<your10digits>' order by a.last_seen_at desc nulls last;"
pm2 logs <pm2-name> --lines 100 --nostream | grep -i identity
```

> **You should see:** the app you used has a `last_seen_at` from the last minute. The `grep` prints nothing.

> **If not:** do not move to the next app until it is fresh. Do not filter the query by `app_key` (that hides the other apps).

3.3 ScoreIT only: check the login fix first

An SMS is sent. `<port>` is ScoreIT's `PORT`.

```shell
curl -s -X POST localhost:<port>/api/auth/send-otp -H 'content-type: application/json' -d
'{"phoneNumber":"<your10digits>"}'
curl -s -X POST localhost:<port>/api/auth/otp-verify -H 'content-type: application/json' -d
'{"phoneNumber":"<your10digits>","otp":"000000"}'
```

> **You should see:** the second answer is `{"error":"OTP invalid"}` (or "invalid or expired") and has **no `accessToken`**.

> **If not:** if you get a token, the fix is not deployed. **Stop. Do not continue with ScoreIT.** Then check that the real code from the SMS does return a token.

3.4 Notes for each app

- **PredictIt:** also set `SCOREIT_API_URL` to ScoreIT's API address (PredictIt copies ScoreIT matches). A newer, **untested** branch `feat/identity-sso-login` exists. Do not deploy it unless you want login-once across apps and have tested it.
- **TurfIT and GameIT:** merge the identity code and `fix/identity-skip-data-avatar`, then follow the recipe.
- **Sponsor (optional):** make sure `AUTH_SECRET` is set; make sure its database has `sp_sponsor_auth_users.mobile_number`; fill `SRC_SPONSOR_URL` and `LINK_SPONSOR_URL`; repeat Steps 2.4 to 2.6; then the recipe with key `sponsor`. Only OTP logins link, email and password logins do not.

3.5 After all apps: the service-down test (once)

```shell
pm2 stop identity
```

Log in on one app. **It must still work.** Its log shows `[Identity] linking … failed`. Then:

```shell
pm2 start identity
curl http://127.0.0.1:4100/health
```

PART 4. The CheerIT API and SSO

The CheerIT API is the repository `Shriya2020/CheerITWebAPI` (Node, Prisma, PostgreSQL).

Pick your situation:

- **Situation 1:** CheerIT is **already live** and you only add SSO. This is the usual production job. Go to **4.1**.
- **Situation 2:** you must **host a new copy** of the CheerIT API (this is how the dev server was built). Go to **4.2**, and then do 4.1 steps 3 to 7.

4.0 Which code to use

The SSO change is **one commit**: `d5f3b67` (5 files, 181 lines added, nothing removed). It adds the SSO route and links users to the identity.

- Cleanest choice: branch **`sso-login`** (that one commit on top of `main`). The project is in the folder `CheerITWebAPIV4/`.
- The dev server currently uses branch **`globaldb`**. There, the project is at the **top of the repository** (no `CheerITWebAPIV4` folder).
- **Do not blindly merge `globaldb`:** it contains a commit with a committed `.env`. Check the `.env` after any checkout.

Below, "project folder" means the folder that contains `package.json` and `src`.

4.1 Situation 1: add SSO to the CheerIT that is already running

Step 1. Back up CheerIT's database (Step 2.1).

Step 2. Deploy the branch and install

```shell
cd <project-folder>
npm install
psql "postgresql://<owner>@<db-host>:5432/<cheerit_db>" -f identity_link.sql
npx prisma generate
```

`identity_link.sql` only adds empty columns. It is safe to repeat. **Never** use `prisma migrate` or `db push` here.

Step 3. Put the two SSO settings in CheerIT's `.env`

Use Step C of the recipe in section 3.2 with **key name** `com` and `<backend-folder>` = the CheerIT project folder. Use the identity service's private address if CheerIT runs on another server.

Step 4. Restart

```
SHELL

pm2 restart <cheerit-pm2-name> --update-env
```

Step 5. Check SSO (this uses a deliberately wrong token)

```
SHELL

curl -s -o /dev/null -w "%{http_code}\n" -X POST https://<cheerit-api>/api/auth/sso-login -H
'content-type: application/json' -d '{"token":"wrong"}'
```

> **You should see:** `401` .

> **If not:** `503` means the two settings are missing or not loaded. `404` means the SSO commit is not deployed.

Step 6. What SSO does (tell CheerIT's team)

ScoreIT proves who the viewer is. The identity service gives a one-time token (valid 5 minutes). CheerIT swaps it for a normal web session. **If the person has no CheerIT account, one is created** (name from the identity, gender `OTHER`). Blocked users get 403. Ask CheerIT's team to confirm that creating accounts this way is acceptable.

Step 7. Sync CheerIT's existing users into the identity

Fill `SRC_COM_URL` and `LINK_COM_URL` in the identity `.env` (Step 2.3), then repeat Steps 2.4, 2.5 and 2.6. CheerIT now appears as another app. Optional: users also get linked at their first SSO login.

4.2 Situation 2: host a new CheerIT API (the dev-server method)

Server: Amazon Linux 2023, user `ec2-user` , nginx, pm2, Node 22 or newer. Replace `<domain>` , `<port>` , `<project-folder>` .

4.2.1 Prepare

- **DNS:** an A record for `<domain>` must point to the server. Check: `dig +short <domain>` shows the server's public IP.

- **Free port:** `ss -ltnp | grep -E ':(3000|3001|3002|3003|4100)'` and pick one that is not listed, for example `3002`.
- **Redis is required.** CheerIT waits for Redis when it starts, even though `REDIS_ENABLED=false`. Without it, the app never starts listening.

```shell
sudo dnf install -y redis6
sudo systemctl enable --now redis6
redis6-cli ping
```

> **You should see:** `PONG`. (Amazon Linux 2: `sudo amazon-linux-extras install -y redis6`, service name `redis`.)

4.2.2 Get the code

```shell
cd ~/websites
git clone https://github.com/Shriya2020/CheerITWebAPI.git
cd CheerITWebAPI
git checkout <branch>
cd <project-folder>
git update-index --skip-worktree .env
npm install
```

`skip-worktree` stops later `git pull`s from replacing the server's `.env`.

> **If not:** if it says `Unable to mark file`, the path to `.env` is wrong for this branch. On `globaldb` the project folder is the repository top, on `sso-login` it is `CheerITWebAPIV4/`.

4.2.3 Create the database

Use the server's own Postgres. Create a **separate** database for CheerIT. Never change the password of a shared admin user.

```shell
createdb -h localhost -p 5432 -U postgres cheeritback
```

Create the tables from CheerIT's schema. This is the **one place** `prisma db push` is right: a **brand-new empty** database. Make sure `DATABASE_URL` points at the **new**

database first.

```shell
npx prisma db push
```

A new empty CheerIT has no reference rows and cannot work without them. Load them **only on a new empty database, never on production** (production already has real rows):

```shell
psql -h localhost -p 5432 -U postgres -d cheeritback <<'SQL'
insert into "EventStatus"(eventstatus_id, code, name) overriding system value values
  (1,'UPCOMING','Upcoming'),(2,'SCHEDULED','Scheduled'),(3,'LIVE','Live'),
(4,'COMPLETED','Completed')
  on conflict do nothing;
insert into "EventCategory"(event_category_id, code, name, is_sport, updated_at)
  values ('f7b3e5e6-a5f6-4aa8-9376-6384811c0165','SPORT','Sport',true,now()) on conflict do
nothing;
insert into "EventType"(eventtype_id, code, name, event_category_id, updated_at) overriding system
value values
  (1,'CRICKET','Cricket','f7b3e5e6-a5f6-4aa8-9376-6384811c0165',now()),
  (2,'FOOTBALL','Football','f7b3e5e6-a5f6-4aa8-9376-6384811c0165',now()),
  (8,'BADMINTON','Badminton','f7b3e5e6-a5f6-4aa8-9376-6384811c0165',now())
  on conflict do nothing;
insert into "Organization"(org_id, code, name, type, updated_at)
  values ('11111111-1111-4111-8111-111111111111','SCOREIT','ScoreIT','OTHER',now()) on conflict do
nothing;
insert into "SMSSetting"(sms_setting_id, provider, is_active, updated_at)
  select gen_random_uuid(),'webtech',true,now() where not exists (select 1 from "SMSSetting");
insert into "SMSTemplate"(sms_template_id, code, message, is_active, updated_at)
  values (gen_random_uuid(),'LOGIN_OTP','Your OTP is {#var#}. Do not share it with anyone.
AvenCheer',true,now())
  on conflict (code) do nothing;
SQL
```

Check the rows are there:

```shell
psql -h localhost -p 5432 -U postgres -d cheeritback -c 'select (select count(*) from
"EventStatus") as statuses, (select count(*) from "EventType") as types, (select count(*) from
"Organization") as orgs, (select count(*) from "SMSSetting") as sms, (select count(*) from
"SMSTemplate") as templates'
```

> **You should see:** `4, 3, 1, 1, 1`. The OTP text must match the text registered with the SMS provider. If you want real data instead, restore a `pg_dump` of an existing CheerIT database into the new one.

Then:

```shell
psql -h localhost -p 5432 -U postgres -d cheeritback -f identity_link.sql
npx prisma generate
```

4.2.4 Edit CheerIT's `.env` before the first start

The `.env` that comes from Git may hold another environment's settings, **including production**. Open it with `nano .env` for any line with a password. Check these lines one by one:

- `DATABASE_URL` = `postgresql://<user>:<pw>@localhost:5432/cheeritback?schema=public`. **It must never point at the production database.** Check without showing the password: `grep '^DATABASE_URL=' .env | sed -E 's#(://[^:/@]+):[^@]*@#\1:<pw>@#'`
- `PORT` = the free port, for example `3002`
- `BASE_URL`, `APP_PUBLIC_URL`, `SQUAD_INVITE_BASE_URL` = `https://<domain>`
- `WEB_BASE_URL` = the CheerIT website address for this environment
- `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` = **new random values**, never another environment's. Create them without showing them:

```shell
sed -i "s|^JWT_ACCESS_SECRET=.*|JWT_ACCESS_SECRET=$(openssl rand -hex 48)|" .env
sed -i "s|^JWT_REFRESH_SECRET=.*|JWT_REFRESH_SECRET=$(openssl rand -hex 48)|" .env
```

- `IDENTITY_SERVICE_URL`, `IDENTITY_API_KEY` = set with the recipe Step C using key name `com` (replace any value that came from Git)
- `REDIS_URL` = `redis://127.0.0.1:6379`
- `AWS_REGION`, `S3_BUCKET`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` = the cheer upload bucket and **fresh** keys that may write to it. **Do not set** **`AWS_ENDPOINT_URL_S3`** (that is only for local testing).
- `SMS_AUTH_KEY` and the other `SMS_*` lines = the SMS provider's values
- `APPLE_REVIEW_ENABLED` = `false` in production (it is a fixed-code login for one number)

4.2.5 Start it

```SHELL
pm2 start src/server.js --name <cheerit-pm2-name>
pm2 save
pm2 logs <cheerit-pm2-name> --lines 30 --nostream
```

> **You should see:** `Prisma connected successfully` and then `Server running on http://localhost:<port>`.

> **If not:** if it stops after the Prisma line, Redis is not running (4.2.1).

Check it answers:

```SHELL
curl -s http://127.0.0.1:<port>/api/events/mobile/live
```

> **You should see:** `{"success":true,…`

Optional: the moderation worker `src/worker/Worker.js` is only needed for events that use CheerIT's AI check: `pm2 start src/worker/Worker.js --name <cheerit-pm2-name>-worker`.

4.2.6 nginx (the web address)

Create the file:

```SHELL
sudo nano /etc/nginx/conf.d/<domain>.conf
```

Paste this, replacing `<domain>` and `<port>`, then save (Ctrl+O, Enter, Ctrl+X):

```NGINX
server {
    listen 80;
    server_name <domain>;
    client_max_body_size 60m;

    location / {
        proxy_pass http://127.0.0.1:<port>;
```

```
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

SHELL

```
sudo nginx -t
sudo systemctl reload nginx
curl -s http://<domain>/api/events/mobile/live | head -c 150
```

You should see: `nginx -t` says the syntax is ok, and the `curl` returns `{"success":true…` .

If not: if you get nginx's own 404 page, the file was not loaded yet (reload and try again) or another site catches the name. Look at what nginx loaded: `sudo nginx -T | grep -n -B2 -A12 '<domain>'` .

4.2.7 HTTPS (required)

Browsers block uploads from a secure page to an insecure address, and the CheerIT website turns `http://` media links into `https://` .

SHELL

```
sudo certbot --nginx -d <domain>
curl -s https://<domain>/api/events/mobile/live | head -c 150
```

Choose "redirect http to https" when asked. The certificate renews itself.

4.2.8 Finish

Now do steps 3 to 7 of section 4.1 (the `com` key, restart, the 401 check, the user sync).

4.3 Storage permission for cheer uploads

The viewer's browser uploads **directly to the bucket**. The bucket must allow that from ScoreIT's website. In the bucket's CORS settings:

JSON

```
[{
  "AllowedOrigins": ["https://<score-website>", "https://<cheerit-website>"],
  "AllowedMethods": ["POST", "GET", "HEAD"],
  "AllowedHeaders": ["*"],
  "ExposeHeaders": ["ETag"],
  "MaxAgeSeconds": 3000
}]
```

Test it before any cheer:

SHELL

```
curl -s -i -X OPTIONS "https://<bucket>.s3.<region>.amazonaws.com/" -H "Origin: https://<score-website>" -H "Access-Control-Request-Method: POST" | grep -i access-control-allow-origin
```

You should see: your website address printed. After one test cheer: `aws s3 ls s3://<bucket>/non-ai-approved/ --recursive | tail -3` shows the new file. The website, the CheerIT API and the bucket must **all** be https.

PART 5. The ScoreIT cheering feature

Branch `feat/cheerit-cheer` of `Shriya2020/ScoreIT` (it already includes the login fix). **Do Part 4 first.**

5.1 Decide these three things first

1. **SSO or a CheerIT code once?** Recommended: **SSO** (`VITE_CHEERIT_TOKEN_LOGIN=on`).
2. **Public or private events?** Recommended: **private** at first. **Public** events show in CheerIT's own live list **and send a "live now" notification to every CheerIT user for every match**.
3. **May ScoreIT create events, teams and competitions in CheerIT?** Recommended: yes, but test with fake team names first. (CheerIT's create routes have no login today.)

5.2 ScoreIT backend

```shell
cd <score-folder>/backend
npm install
npx prisma generate
```

Add these lines to its `.env` (check for repeated lines):

- `IDENTITY_SERVICE_URL` , `IDENTITY_API_KEY` : already set in Part 3.
- `CHEERIT_API_URL=https://<cheerit-api>` (no `/api` at the end)
- `CHEERIT_CREATE_EVENTS=on` (**off by default**; it turns on event creation, logos and score sending)
- `CHEERIT_ORG_ID=<organization id>` : only needed the first time, to create the "ScoreIT" competitions. Not needed if CheerIT's admin created the competitions "ScoreIT", "ScoreIT Football" and "ScoreIT Badminton".
- `CHEERIT_EVENT_ACCESS=private` (or `public`)
- `CHEERIT_EVENT_TAG=ScoreIT` (use `ScoreIT-dev` on a dev ScoreIT that feeds the same CheerIT, so match numbers cannot collide)
- `CHEERIT_SYNC_SECONDS=15` (default)
- `CHEERIT_EVENT_HOURS=12` (default; must be longer than a match)
- `SCORE_PUBLIC_URL=<address where ScoreIT serves uploaded files>` (to copy uploaded team logos)

- Optional: `CHEERIT_VENUE_ID`, `CHEERIT_COMPETITION_ID`, `CHEERIT_GROUP_ID`, `CHEERIT_EVENTTYPE_ID`, `CHEERIT_VIDEO_SECONDS`

Restart:

```shell
pm2 restart scoreit-backend --update-env
pm2 logs scoreit-backend --lines 30 --nostream
```

> **You should see:** `creating CheerIT events and sending scores for live matches every 15s`, and **no** repeating `background check failed`.

5.3 ScoreIT website

The website reads these settings **when it is built**. Set them in the build environment, then build again.

- `VITE_CHEERIT_API_URL=https://<cheerit-api>`
- `VITE_CHEERIT_TOKEN_LOGIN=on`
- Optional: `VITE_CHEERIT_WEB_URL`, `VITE_PREDICT_API_URL`, `VITE_PREDICT_WEB_URL`

```shell
cd <score-folder>/frontend
npm install
npm run build
```

Then copy the build to the folder nginx serves (on dev: `/var/www/scoreit`). `npm install` is required because the website now needs a library called `mediabunny` (tested version 1.55.2).

If the ScoreIT server has uncommitted hand-edited files (the dev server does), do not `git pull` over them. First make a safe copy:

```shell
rsync -a --exclude node_modules --exclude dist <score-folder>/ <score-folder>.backup-<date>/
```

Then look at `git status` and bring the branch's files in on purpose.

5.4 Check it works

```shell
curl -s http://127.0.0.1:<score-port>/api/public/matches/<live-match-id>/cheerit
```

> **You should see:** `{"linked":true,"eventId":…,"eventName":"A vs B (ScoreIT #n)"}` . **If you see a reason instead:**

- `no_match` : creation is off, the match is not live, or the sport is not supported (only cricket today).
- `unavailable` : ScoreIT cannot reach CheerIT. Check `CHEERIT_API_URL` and look for a repeated line in `.env` .
- `not_configured` : `CHEERIT_API_URL` is not set.

5.5 What the feature does

- Each live ScoreIT match gets **its own CheerIT event**, named like `A vs B (ScoreIT #12)` . A team missing in ScoreIT data is shown as "Unknown".
- Teams are found by name or created. They get the uploaded ScoreIT logo, or the same default picture the ScoreIT website shows.
- Every 15 seconds ScoreIT sends the ball-by-ball score to CheerIT. Only differences are sent. Edits and undos are copied. The scorecard is marked complete when the match ends.
- The viewer: picks a team, uses the **full-screen camera** (photo or video up to 7 seconds), passes a **sound check** (refused if shorter than 2 seconds, silent or too quiet), the video is **compressed in the browser** (about 2 MB), sees a preview with **Retake**, agrees, and uploads.
- Login is SSO. If SSO fails the card asks for a CheerIT code instead.
- The cheer is saved in CheerIT at once (**APPROVED**, unless the event has CheerIT's AI check on). A person can cheer up to **5 times per event**, counting cheers from ScoreIT and from the CheerIT app together.
- If events are **public**, people can also cheer on them from the CheerIT app and website.

5.6 Optional: team name and score on the CheerIT website card

Repository `Shriya2020/CheerIT-COM` , branch `feat/card-team-name-and-score` (two files: `src/api/client.ts` and `src/components/MatchCard.tsx`).
Merge it the normal way. The website takes CheerIT's address from three settings that

must all match: `VITE_API_URL`, `VITE_DIRECT_API_BASE_URL`, `VITE_PROXY_API_URL`.

PART 6. Final checks

Tick each one. All must pass.

- [ ] `curl http://127.0.0.1:4100/health` returns `{"ok":true}`
- [ ] `dry-run`, `apply` and `link --verify-only` all PASS, and the second `--apply` writes 0
- [ ] ScoreIT wrong-code test (3.3): no token
- [ ] Log in on each app with your own number: its `last_seen_at` is fresh
- [ ] `pm2 stop identity`, then log in: login still works. Then `pm2 start identity`
- [ ] CheerIT `sso-login` with a wrong token returns `401`
- [ ] The CheerIT API address over https returns `{"success":true…`
- [ ] A fake live cricket match in ScoreIT: within 15 seconds an event "…(ScoreIT #n)" exists in CheerIT
- [ ] The ScoreIT page for that match shows **Photo and Video** buttons
- [ ] Cheer once from a phone you own, with SSO: no code is asked, and the cheer is saved
- [ ] The file is in the bucket (`aws s3 ls …/non-ai-approved/ --recursive`)
- [ ] The same cheer is visible and plays in the CheerIT app or website
- [ ] The fake match's rows are deleted from CheerIT's admin

If something goes wrong

How to undo things

- **One app off:** remove its two `IDENTITY_` lines from its `.env`, then `pm2 restart <name> --update-env`. Login works as before.
- **Identity off for everyone:** `pm2 stop identity`. Apps keep working and log a warning per login.
- **Stop creating events and scores:** remove `CHEERIT_CREATE_EVENTS` from ScoreIT's `.env` and restart ScoreIT. Existing events stay.
- **Back to a code login in the card:** remove `VITE_CHEERIT_TOKEN_LOGIN` and rebuild the website.
- **Hide the card:** remove `VITE_CHEERIT_API_URL` and rebuild.
- **SSO off in CheerIT:** remove its two `IDENTITY_` lines and restart. The route then answers 503 and the card falls back to a code login.
- **Delete the identity data:** `DROP DATABASE cheeritidentity;` The `identity_id` columns are empty-able and harmless; leave them.
- **Restore a database:** `pg_restore --clean` from the Step 2.1 backup.

Problems and fixes

The identity scripts

- `dry-run` prints `Skipped` for every app: the `.env` was not loaded. Use `node --env-file=.env …`.
- `database "cheeritidentity" does not exist`: create it (Step 2.2) or fix `IDENTITY_URL`.
- `Set IDENTITY_URL …` when starting: the setting is called `IDENTITY_URL`, not `DATABASE_URL`.
- Key length prints `0`: the key command failed. Run `npm run issue-key -- <app>` and read the error.
- `.env` contains `IDENTITY_API_KEY=<…>`: a placeholder was pasted. Issue a new key and rewrite the line.

An app's log

- `HTTP 401` (`[Identity] linking … failed`): wrong or cancelled key. Issue a new one, replace the line, restart with `--update-env`.

- `HTTP 500` and `PayloadTooLargeError` in `pm2 logs identity`: an app sent a base64 photo. Deploy `fix/identity-skip-data-avatar` (GameIT, TurfIT).
- `HTTP 409`: that user is already linked to a different phone. Fix by hand. The service never moves a user.
- `identityId` Prisma errors: out-of-date Prisma client. Run `npx prisma generate` and restart.

Hosting the CheerIT API

- "no such file or directory" in the CheerIT folder: wrong folder. On `globaldb` the project is the repository top, on `sso-login` it is `CheerITWebAPIV4/`.
- CheerIT stops after `Prisma connected successfully`: Redis is not running (4.2.1).
- `SMS settings not configured` on an OTP request: the new database lacks the SMS row (4.2.3).
- OTP request accepted but no SMS arrives: the `LOGIN_OTP` text must match the text registered with the SMS provider: "Your OTP is {#var#}. Do not share it with anyone. AvenCheer".
- nginx's own 404 for the domain: reload nginx and retry, or check `sudo nginx -T | grep -n -B2 -A12 '<domain>'`.
- `sso-login` answers 503: the two `IDENTITY_` settings are missing or not loaded (use `--update-env`). It answers 404: the SSO commit is not deployed.

ScoreIT and cheering

- `POST /api/auth/cheerit-sso` 502 in ScoreIT's log: the identity service is down or unreachable. The card falls back to a code login.
- ScoreIT log repeats `background check failed: fetch failed`, or the lookup says `unavailable`: `CHEERIT_API_URL` points at something that is down, or a repeated line in `.env` overrides it.
- The card shows only photo buttons: the CheerIT event was saved with video off. CheerIT's **create** route reads `allow_video_audio` (ScoreIT commit `03e0dc53` sends it). For an old event: `update "Event" set allow_video_with_audio = true where …`
- 5 live matches show fewer events: old behaviour (one per pair of teams). Deploy ScoreIT `867ef9ae` or later.
- A match gets a second event after some hours: events stay open `CHEERIT_EVENT_HOURS` (12). Keep it longer than a match.
- The browser upload fails even though a slot was given: the bucket has no CORS rule for the website (4.3), or something is not https.
- `ERR_SSL_PROTOCOL_ERROR` on pictures or videos: the link points at a plain-http address. This only happens with a local test storage; real S3 is https.

- Team pictures broken on the CheerIT website: CheerIT's `BASE_URL` must be an https address.
- "team_id … not allowed for non-sports events": the CheerIT event is not a sports type. Only CheerIT's team can change that.
- The camera does not open: the page must be https (or localhost) and the browser must allow the camera. The card then offers "Use camera app".
- The predict card is empty: PredictIt is not reachable at `VITE_PREDICT_API_URL`.

Reference

Where each setting goes

`cheeritidentity/.env`: `IDENTITY_URL`, `PORT`, `HOST` (optional), `SRC_*_URL` (read-only), `LINK_*_URL` (owner). See Step 2.3.

Every app backend `.env` (ScoreIT, GameIT, PredictIt, TurfIT, Sponsor): `IDENTITY_SERVICE_URL` (`http://127.0.0.1:4100`, or the identity service's private address) and `IDENTITY_API_KEY` (that app's own key).

CheerIT API `.env`: `IDENTITY_SERVICE_URL`, `IDENTITY_API_KEY` (the `com` key); plus `DATABASE_URL`, `PORT`, `BASE_URL`, `APP_PUBLIC_URL`, `SQUAD_INVITE_BASE_URL`, `WEB_BASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `REDIS_URL`, `AWS_REGION`, `S3_BUCKET`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `SMS_*`. See 4.2.4.

ScoreIT backend `.env`: `CHEERIT_API_URL`, `CHEERIT_CREATE_EVENTS`, `CHEERIT_ORG_ID`, `CHEERIT_EVENT_ACCESS`, `CHEERIT_EVENT_TAG`, `CHEERIT_SYNC_SECONDS`, `CHEERIT_EVENT_HOURS`, `SCORE_PUBLIC_URL`, `LOG_OTP_TO_CONSOLE=false` (production). See 5.2.

ScoreIT website build environment: `VITE_CHEERIT_API_URL`, `VITE_CHEERIT_TOKEN_LOGIN`, `VITE_CHEERIT_WEB_URL`, `VITE_PREDICT_API_URL`, `VITE_PREDICT_WEB_URL`. See 5.3.

PredictIt backend `.env`: `SCOREIT_API_URL` (ScoreIT's API address), `OTP_DEV_MODE=false`, `OTP_LOG_CODE=false` (production).

CheerIT website build environment: `VITE_API_URL`, `VITE_DIRECT_API_BASE_URL`, `VITE_PROXY_API_URL`: all three are CheerIT's API address.

CheerIT storage bucket: the CORS rule in 4.3.

Addresses used on the dev server (for orientation only)

- CheerIT API: `https://devcheerback.avenengage.com` (nginx to port 3002, pm2 `cheeritwebapi-dev`, database `cheeritback`)
- ScoreIT: `https://devscoreit.avenengage.com` (backend port 3001, pm2 `scoreit-backend`, website built into `/var/www/scoreit`)
- Identity service: `127.0.0.1:4100` (pm2 `identity`)
- CheerIT website (dev): `devcheerit.avenengage.com`

Production addresses are different. Fill them in using section 4.

Where the code is

- `kartikavencheer/cheeritidentity`, branch `main`: the identity service and scripts, these documents, test scripts in `dev-tools/`
- `Shriya2020/ScoreIT`, branch `feat/cheerit-cheer`: ScoreIT identity code, login fix, cheer card, events, logos, scores, video and sound logic
- `Shriya2020/CheerITWebAPI`, branch `sso-login` (clean) or `globaldb`: the SSO route (commit `d5f3b67`)
- `Shriya2020/CheerIT-COM`, branch `feat/card-team-name-and-score`: team name and score on the card
- `Shriya2020/predictitapp`: `master` (identity code); `feat/identity-sso-login` is optional and untested
- `Shriya2020/pickleball` and `Shriya2020/turfitweb`: `fix/identity-skip-data-avatar`
- `Shriya2020/CheerIT-Sponsor`: `stackchnage`

Other files in this repository (background only): `CHEERIT_PRODUCTION_GUIDE.md` (the cheer feature in depth), `PRODUCTION_RUNBOOK.md` and `PRODUCTION_COMMANDS.md` (earlier versions of Parts 2 and 3), `HANDOFF.md` (status), `CONVERSATION.md` (history).